Lab 1: Bayesian Networks

Your Name

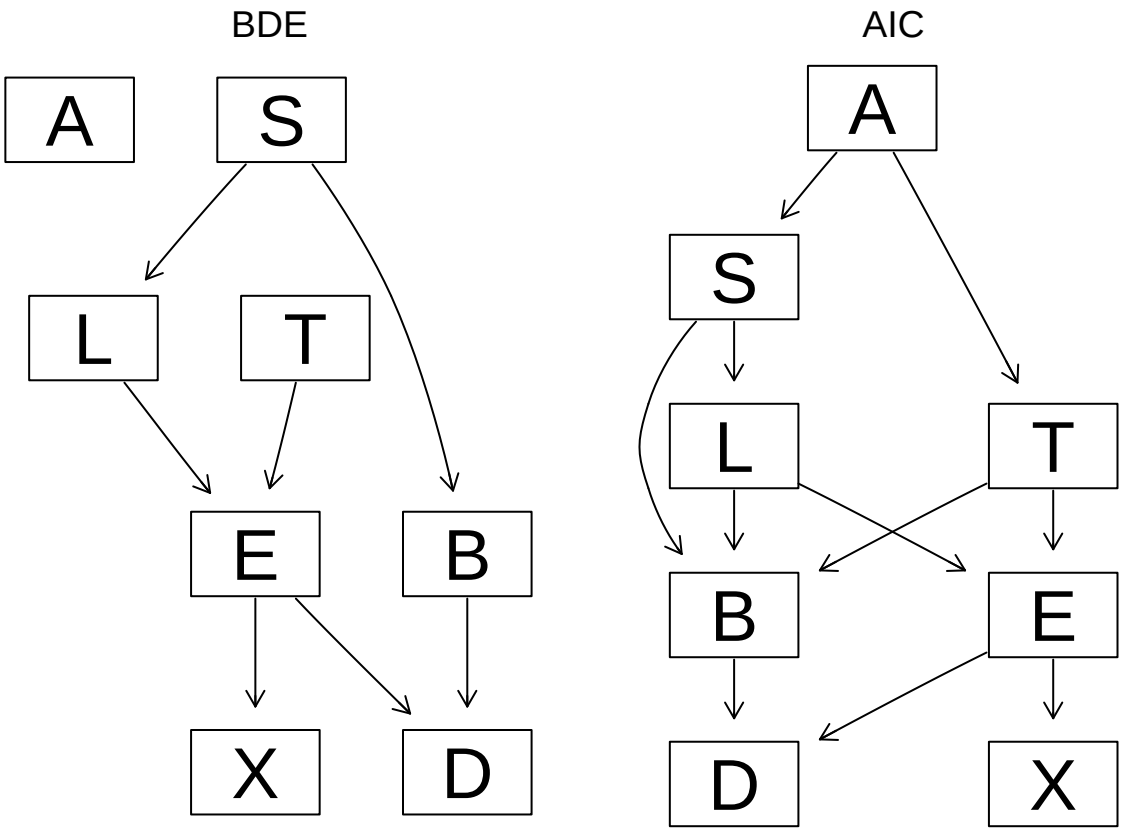
2026-09-14

Question 1

Show that multiple runs of the hill-climbing algorithm can return non-equivalent Bayesian network (BN) structures. Explain why this happens.

Hill Climbing with Different Scores

```
dag1 <- hc(asia, score="bde")
dag2 <- hc(asia, score="aic")
par(mfrow=c(1,2), mar=c(1,1,2,1))
graphviz.plot(dag1, main="BDE")
graphviz.plot(dag2, main="AIC")
```




```
all.equal(cpdag(dag1), cpdag(dag2))
```

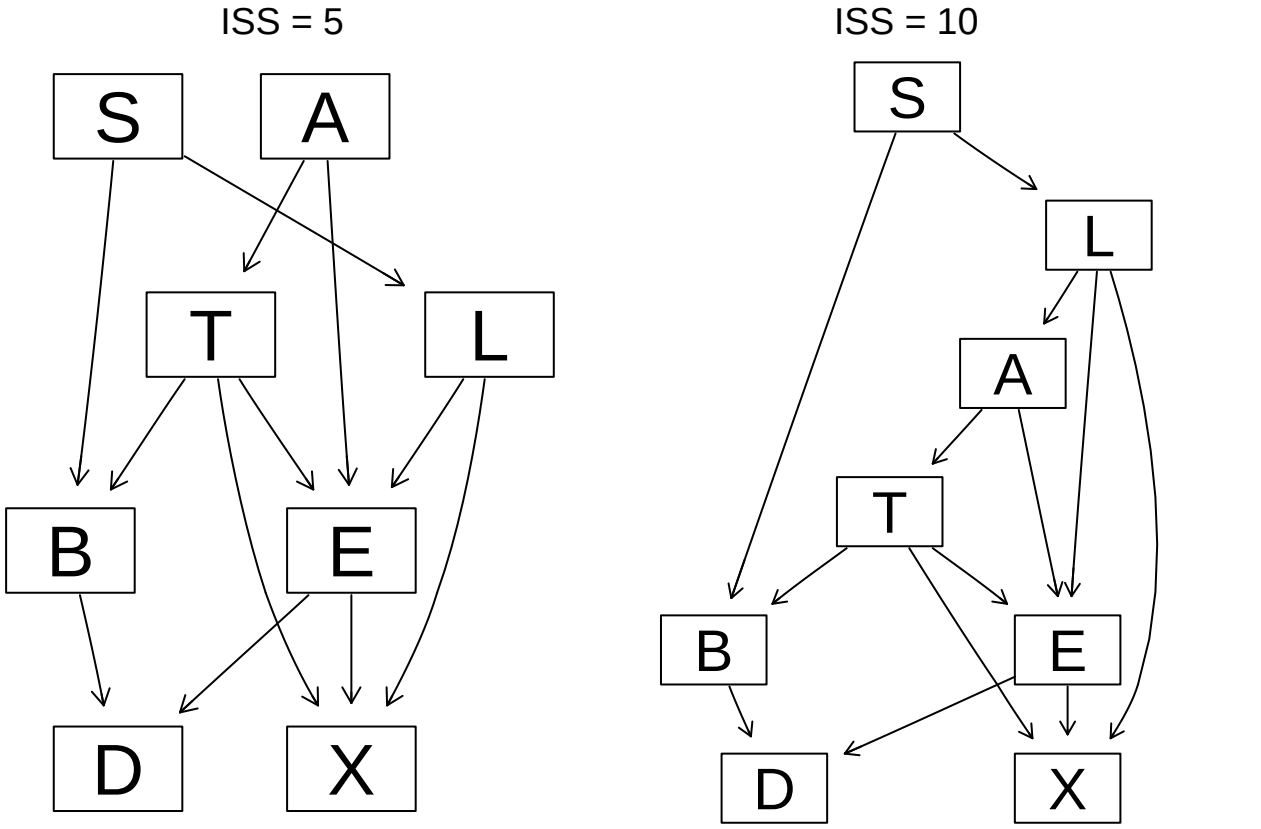
```
## [1] "Different number of directed/undirected arcs"
```

```
score(dag1, asia, type="bde") == score(dag2, asia, type="bde")
```

```
## [1] FALSE
```

Hill Climbing with Different ISS

```
dag1 <- hc(asia, score="bde", iss=5)
dag2 <- hc(asia, score="bde", iss=10)
par(mfrow=c(1,2), mar=c(1,1,2,1))
graphviz.plot(dag1, main="ISS = 5")
graphviz.plot(dag2, main="ISS = 10")
```

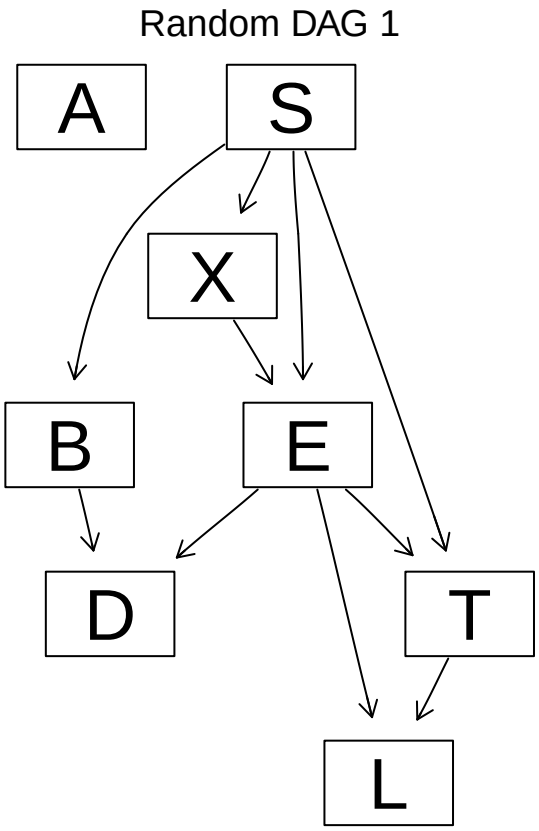


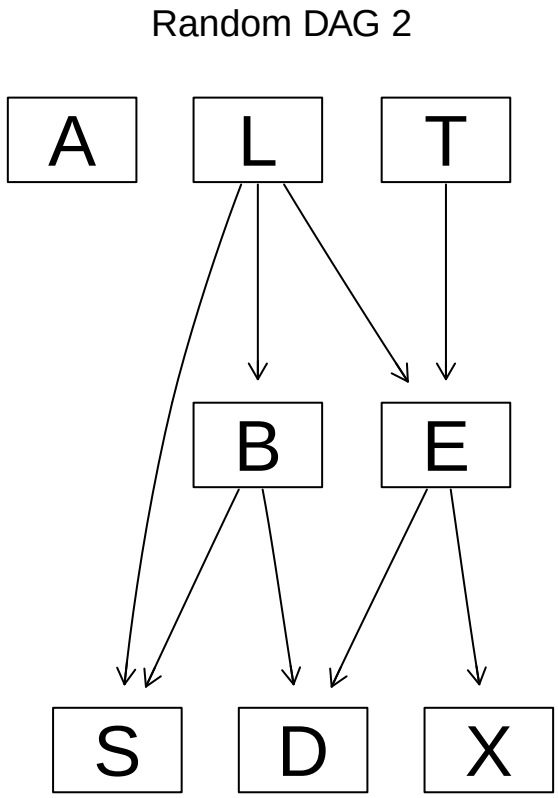

```
all.equal(cpdag(dag1), cpdag(dag2))
```

```
## [1] "Different number of directed/undirected arcs"
```

Hill Climbing with Random Initial Structures

```
set.seed(12345)
dag1 <- hc(asia, score="bde", start=random.graph(colnames(asia)))
dag2 <- hc(asia, score="bde", start=random.graph(colnames(asia)))
par(mfrow=c(1,2), mar=c(1,1,2,1))
graphviz.plot(dag1, main="Random DAG 1")
graphviz.plot(dag2, main="Random DAG 2")
```






```
all.equal(cpdag(dag1), cpdag(dag2))
```

```
## [1] "Different number of directed/undirected arcs"
```

Using different score methods, imaginary sample size, or initial structures when running the hill climb algorithm yields non-equivalent Bayesian networks. This is shown by the plots and by converting the DAGs to equivalence classes by using *cpdag*() function and comparing them using *all.equal*(). This checks if two DAGs belong to the same equivalence class.

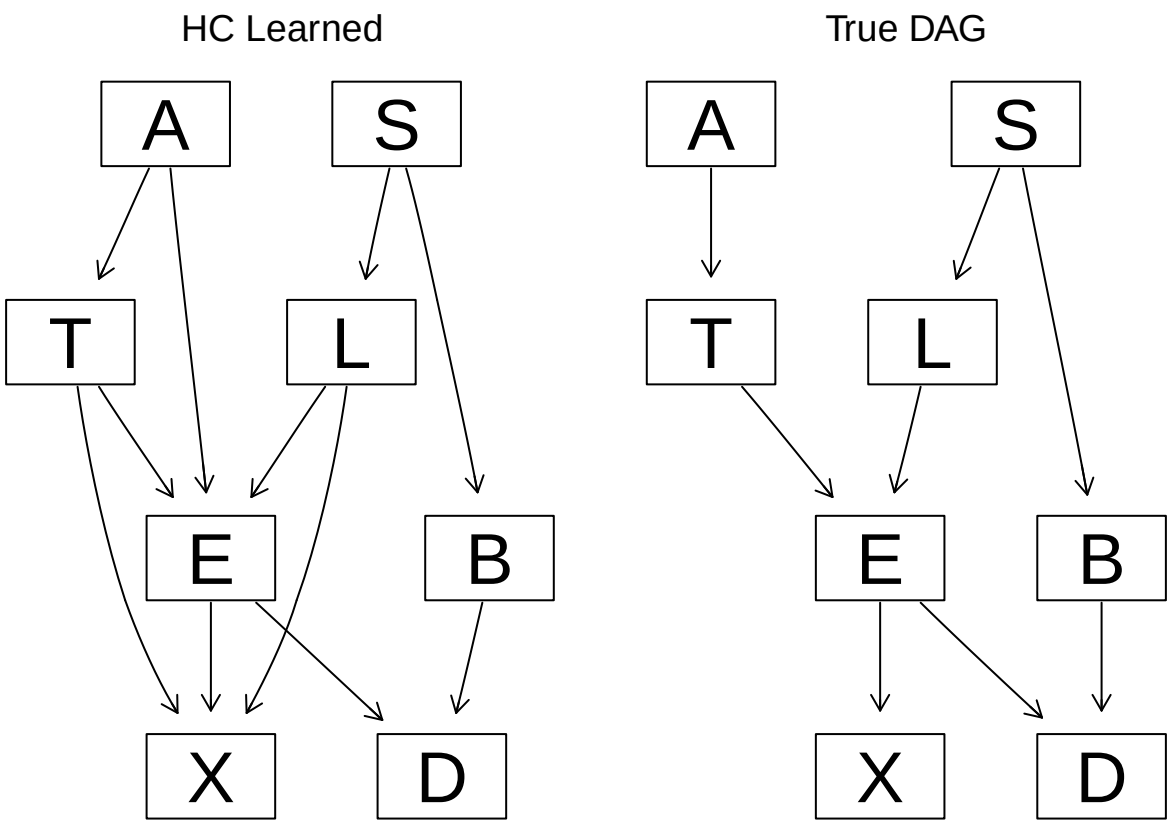
The reason why hill climbing algorithm can yield non-equivalent structures is because it is a greedy algorithm which can and often will get stuck in local optima.

Question 2

Learn a BN from 80 % of the Asia dataset. Use the BN learned to classify the remaining 20 % of the Asia dataset in two classes: S = yes and S = no. In other words, compute the posterior probability distribution of S for each case and classify it in the most likely class.

```
n <- dim(asia)[1]
set.seed(12345)
id <- sample(1:n, floor(n*0.8))
train <- asia[id, ]
test <- asia[-id, ]

hc_dag <- hc(train, score="bde", iss=5)
true_dag <- model2network("[A][S][T|A][L|S][B|S][D|B:E][E|T:L][X|E]")
par(mfrow=c(1,2), mar=c(1,1,2,1))
graphviz.plot(hc_dag, main="HC Learned")
graphviz.plot(true_dag, main="True DAG")
```



Parameter Estimation

```
hc_dag_fit <- bn.fit(hc_dag, data=train, method="mle")
true_dag_fit <- bn.fit(true_dag, data=train, method="mle")

hc_grain <- as.grain(hc_dag_fit)
true_grain <- as.grain(true_dag_fit)
```

Classification

This function classifies the test data set. It takes a gRain model and a set of observed node names, then for each test case computes the posterior probability of S given the observed evidence, predicting "yes" if $P(S = \text{yes}) > 0.5$ and "no" otherwise.

```
classifier <- function(model, observed_nodes) {
  preds <- numeric(nrow(test))
  for(i in 1:nrow(test)) {
    states <- as.matrix(test[i, observed_nodes])
    evidence <- setEvidence(model, nodes=observed_nodes, states=states)
    pred <- querygrain(evidence, nodes="S")$S
    preds[i] <- ifelse(pred["yes"] > 0.5, "yes", "no")
  }
  return(preds)
}
```

```r
nodes <- colnames(test[-2])

hc_model_preds <- classifier(hc_grain, nodes)
table(test$S, hc_model_preds)
```

```
##      hc_model_preds
##        no yes
##   no  337 176
##   yes 121 366
```

```r
true_model_preds <- classifier(true_grain, nodes)
table(test$S, true_model_preds)
```

```
##      true_model_preds
##        no yes
##   no  337 176
##   yes 121 366
```

As shown above, the confusion matrices are identical. When predicting S using all remaining nodes as evidence, we observe S's Markov Blanket (node L and B). As can be seen in the plot, both the learned DAG and the true DAG capture the exact same local joint distribution around S, L, and B. In a Bayesian Network, a target node is conditionally independent of all other nodes in the network given its Markov Blanket. So even though some edge configurations (around node $T, E, X,$) differ, they are conditionally independent of S and have no effect on S.

Question 3

Now, you are asked to classify S given observations only for the so-called Markov blanket of S

```r
hc_mb <- mb(hc_dag_fit, node="S")
true_mb <- mb(true_dag_fit, node="S")

hc_mb_preds <- classifier(hc_grain, hc_mb)
table(test$S, hc_mb_preds)
```

```
##      hc_mb_preds
##        no yes
##   no  337 176
##   yes 121 366
```

```r
true_mb_preds <- classifier(true_grain, true_mb)
table(test$S, true_mb_preds)
```

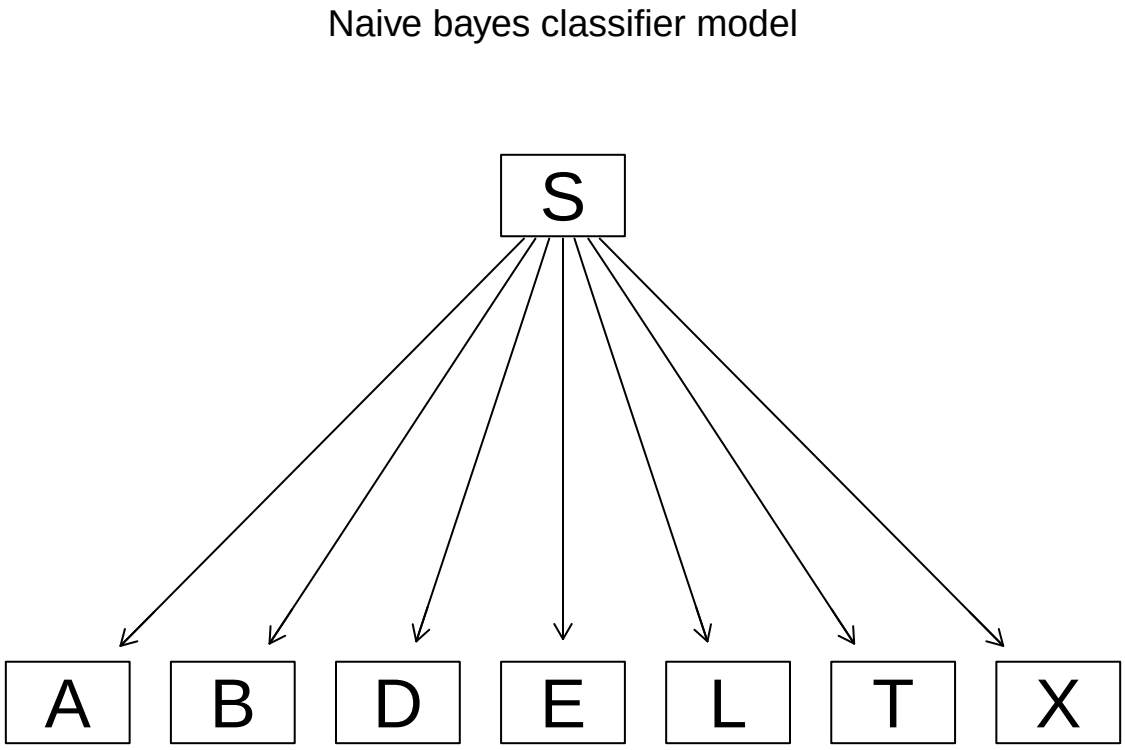
```
##      true_mb_preds
##        no yes
##   no  337 176
##   yes 121 366
```

Here the confusion matrices are also identical. This is because the learned DAG share the same markov blanket as the true DAG, so the evidence outside the markov blanket doesn't affect the posterior probability of S.

Question 4

Repeat the exercise (2) using a naive Bayes classifier

```r
nbc_network <- model2network("[S][A|S][T|S][L|S][B|S][E|S][X|S][D|S]")
graphviz.plot(nbc_network, main = "Naive bayes classifier model")
```




```r
nbc_fit <- bn.fit(nbc_network, data=train, method="mle")
nbc_grain <- as.grain(nbc_fit)

nbc_preds <- classifier(nbc_grain, nodes)
table(test$S, nbc_preds)
```

```
##      nbc_preds
##        no yes
##   no  359 154
##   yes 180 307
```

Question 5

Explain why you obtain the same or different results in the exercises (2-4).

Exercises 2 and 3 produce the same results because both the learned HC DAG and the true DAG share the same Markov blanket for S. By the conditional independence property of BNs, P(S | evidence) depends only on evidence of S's Markov blanket.

Exercise 4 differs because the Naive Bayes classifier assumes all features are conditionally independent given S, removing edges like T→E and B→D. The true DAG has dependencies between features that are lost under the naive Bayes assumption. This assumption leads to different (worse) posterior estimates for S.